
Trabalho de Pesquisa - Colónia de Formigas

Eduardo Junqueira nº 1251561, 1251561@isep.ipp.pt,
João Azevedo nº 1251566, 1251566@isep.ipp.pt,
Rodrigo Martins nº 1250398, 1250398@isep.ipp.pt.
Repositório GitHub: [1].
Supervisionado por Filipe Azevedo, fta@isep.ipp.pt.

Mestrado em Engenharia Electrotécnica e de Computadores
Área de Especialização em Automação e Sistemas



DEPARTAMENTO DE ENGENHARIA ELETROTÉCNICA
Instituto Superior de Engenharia do Porto

Dezembro, 2025

Índice

Lista de Acrónimos	3
Lista de Figuras	4
1 Introdução	2
1.1 Sistemas Naturais Como Fonte de Inspiração	2
1.2 Da Biologia Ao Algoritmo	2
1.3 Objetivo do Relatório	3
2 Desenvolvimento	4
2.1 Princípios do algoritmo Ant Colony Optimization (ACO)	4
2.2 Modelação computacional do comportamento das formigas	4
2.3 Etapas ACO	5
2.3.1 Diagramas Explicativos	6
2.3.2 Pseudocódigo Representativo	7
2.4 (TSP)	8
2.4.1 Ant System (AS)	8
2.4.2 Ant Colony System (ACS)	12
3 Aplicação em Otimização [14]	14
3.1 Parâmetros e "Dataset" Implementado	14
3.2 Resultados:	15
4 Conclusão	16
Referências	18

Lista de Acrónimos

ACO	Ant Colony Optimization
ACS	Ant Colony System
AS	Ant System
Fig	Figura
GA	Genetic Algorithms
TSP	Travelling Salesman Problem
Uc	Unidade Curricular

Lista de Figuras

2.1	Grafo [7].	5
2.2	Diagrama de sobrevivência das formigas [11] [1].	6
2.3	"Flowchart":explicação do pseudocódigo [11], [1].	8
2.4	TSP Assimétrico [11] [1].	9
2.5	Escolha do melhor caminho (TSP) exemplo já implementado! [2].	10
2.6	Possíveis movimentos da formiga $P(x)$, [10].	11
2.7	Variantes de Ant Colony System (ACS) e resultados respetivos [5].	12
3.1	225 cidades de "North-West and Central part of the Indian sub continent". [14]. . .	15

Resumo

Colônia de formigas no contexto de Genetic Algorithms (GA) é o algoritmo de estudo neste relatório. Este algoritmo tem como base os conceitos biológicos das formigas desde a sobrevivência em busca do alimento, reprodução da sua população e comunicação química por substância de feromonas.

Para o Travelling Salesman Problem (TSP), são analisados os principais algoritmos de otimização por colônias de formigas (Ant Colony Optimization (ACO)), incluindo o Sistema de Formigas (Ant System (AS)) com atualização probabilística das feromonas baseada na qualidade das soluções, e o Sistema de Colônia de Formigas (ACS) que incorpora melhorias como a ação *local* de feromonas, pesquisa local com candidato mais próximo, e elitismo para acelerar a convergência.

O problema do TSP, através deste algoritmo visa demonstrar a metodologia da melhor solução. Para este problema será explorado a otimização por colônias de formigas - ACO, no que diz respeito ao sistema de formigas - AS e ao sistema de colônia de formigas - ACS.

As aplicações em otimização neste algoritmo, são expostas como casos de uso aplicáveis para obter uma solução ótima.

Capítulo 1

Introdução

Muitos problemas de engenharia, logística e ciência da computação envolvem a necessidade de encontrar soluções ótimas num vasto espaço de possibilidades, com restrições complexas. Entre estes desafios destaca-se o problemas de otimização combinatória complexos, como o TSP, em que a procura por uma solução "exata" se torna impraticável devido ao crescimento exponencial do número de combinações. Deste modo, métodos inspirados na natureza permitem obter soluções de elevada qualidade com menor custo computacional [2].

1.1 Sistemas Naturais Como Fonte de Inspiração

Apesar das suas capacidades cognitivas individuais serem limitadas, estes insetos (formigas) demonstram a capacidade de desempenhar tarefas complexas através da cooperação indireta, como encontrar rotas eficientes entre o ninho e fontes de alimento. Estudos experimentais, como o conduzido por Goss et al [3], mostraram que este comportamento emerge da interação local entre os indivíduos e do uso de feromonas como mediadores ambientais.

1.2 Da Biologia Ao Algoritmo

Este mecanismo natural serviu de inspiração para o algoritmo ACO, introduzido por Dorigo na década de 1990 [4]. Este algoritmo, traduz matematicamente os princípios biológicos de reforço e evaporação das trilhas de feromonas, permitindo resolver problemas complexos de otimização com base em cooperação indireta entre agentes artificiais. Desde então, o método tem sido aplicado com sucesso em diversos domínios, destacando-se na resolução de problemas de trajetos mínimos, roteamento e planeamento de redes.

1.3 Objetivo do Relatório

O objetivo deste trabalho de pesquisa, consiste no estudo do tema apresentado no título deste relatório: **Colônia de Formigas**. Com base neste estudo, será elaborado um relatório que aborda os principais aspetos do método da Colônia de Formigas, incluindo a **I**: sua inspiração biológica, **II**: o funcionamento do algoritmo e **III**: a sua aplicação.

A sua aplicação, será apresentada em um caso prático de otimização, onde o algoritmo demonstrará o desempenho em problemas como o TSP. Desta forma, o relatório pretende fornecer uma visão completa, desde os fundamentos teóricos até a aplicação prática do método.

A estrutura do relatório é organizada da seguinte forma:

- No Capítulo ?? descreve-se o comportamento natural das formigas e os resultados experimentais observados na literatura.
- No Capítulo 2 é apresentada a formulação matemática do algoritmo ACO e os seus principais mecanismos de funcionamento.
- No Capítulo 3 discutem-se aplicações práticas do algoritmo ACO, com destaque para o problema (TSP), apresentando um caso prático.
- No 4 são apresentadas as conclusões finais.

Capítulo 2

Desenvolvimento

2.1 Princípios do algoritmo ACO

O ACO é uma heurística baseada em probabilidades, publicado por Marco Dorigo em 1992 [5], tendo como ponto de vista resolver problemas computacionais que envolvem a procura do melhor caminho em grafos. Como já foi esclarecido no capítulo ??, este algoritmo traduz matematicamente os princípios biológicos da formiga, em regras de probabilidade e em mecanismos matemáticos aplicáveis a problemas de otimização, é composto por um ciclo que alterna entre a fase inicial de exploração e a fase seguinte de atualização de feromona, até ser atingido um critério suficiente que seja a solução ótima ou uma solução escolhida que muitas vezes não é a solução ótima, mas sim a escolhida por determinados fatores dependendo da utilização deste [6].

2.2 Modelação computacional do comportamento das formigas

As formigas artificiais são agentes computacionais que constroem soluções de forma incremental e com diferenças das formigas biológicas. As formigas artificiais têm alguma memória, não são completamente cegas e vivem num ambiente onde o tempo é discreto [5].

Cada formiga vai construir uma solução passo a passo e vai ser guiada através da feromona e da heurística disponível no problema. Como trabalham em conjunto e de forma autónoma, o algoritmo consegue explorar diferentes caminhos ao mesmo tempo e chegar a soluções mais eficazes.

A feromona é um valor numérico associado às ligações do grafo pela formiga artificial. O valor numérico indica a qualidade das soluções, ou seja, os caminhos com a melhor solução, são as que têm níveis feromona maior. A formiga artificial tende a escolher o caminho com os níveis de feromona mais altos, como ilustrado na Figura (Fig), em baixo, 2.1, e, desta forma, a feromona orienta o comportamento das formigas ao longo das iterações e ajuda o algoritmo a chegar às soluções mais eficazes.

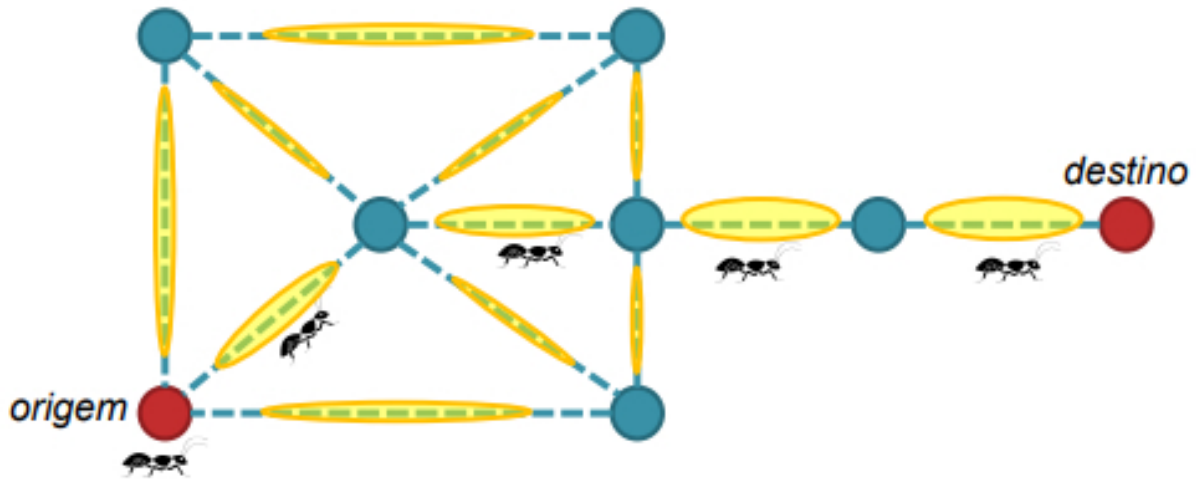


Figura 2.1: Grafo [7].

A Heurística é um valor calculado com base em características imediatas do problema e que ajuda na orientação da decisão da formiga artificial. No início do algoritmo não há feromona suficiente, ou seja, as formigas não podem escolher o caminho completamente ao "acaso". Então, a heurística indica quais são as opções que parecem mais favoráveis naquele momento do problema. No TSP, quando aplicamos o ACS, a heurística mais utilizada é o inverso da distância das cidades ($n_{ij} = 1/d_{ij}$) [8]. Neste caso, os trajetos com as cidades mais próximas vão ter uma heurística maior e as cidades com o trajeto mais longo vão ter uma heurística menor, o que favorece os trajetos mais curtos. Posto isto, a heurística complementa a feromona e contribui para escolhas mais corretas durante a construção da solução.

2.3 Etapas ACO

Após a explicação anterior, 2.1 e 2.2, agora será descrito as principais etapas que compõem o funcionamento do algoritmo de ACO, baseado nas heurísticas e meta-heurística.

A heurística não garante uma solução ótima, mas se tiver bem implementada e com parâmetros bem ajustados, garante uma boa solução a que atribuímos como **solução ótima**. Esta heurística é um algoritmo simples de fórmulas probabilísticas, que aumentam ou diminuem com base nos parâmetros dos problemas, que visam resolver problemas complexos 2.2.

Assim, meta-heurística é uma ciência da computação complexa que procura precisamente uma heurística. Neste caso a meta-heurística em estudo é a colônia de formigas-otimização [9]. Esta é uma heurística das várias meta-heurísticas na qual a Unidade Curricular (Uc) tem: GA [10].

2.3.1 Diagramas Explicativos

A Fig em baixo 2.2, explica de uma forma mais visual a topologia base, presumida para o trabalho de procura de alimentos pelas formigas.

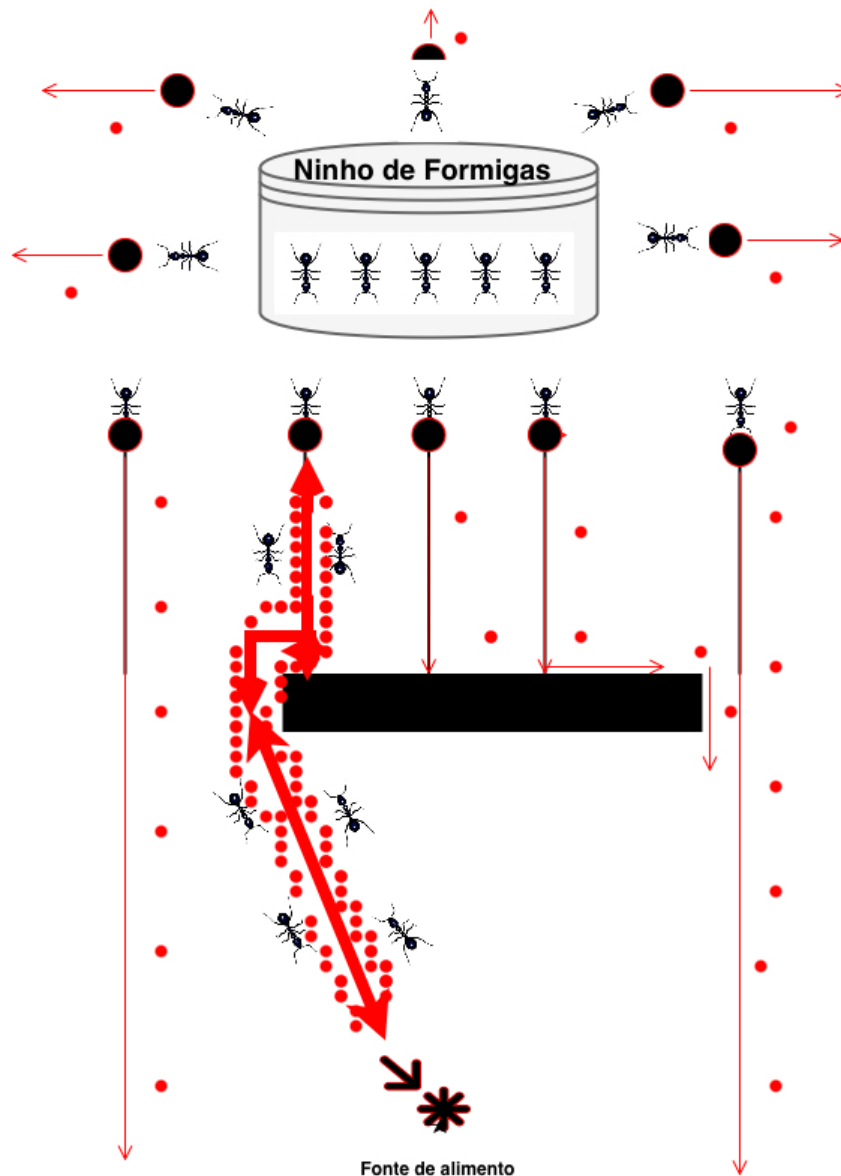


Figura 2.2: Diagrama de sobrevivência das formigas [11] [1].

Cada ponto vermelho, corresponde às feromonas, (são substâncias químicas que funcionam como um sistema de comunicação para orientação e alerta na colónia), de sentido ninho-fonte e fonte-ninho que são libertadas pelas formigas. Verifica-se que o caminho mais curto com a menor distância é o que está com maior número de feromonas logo é o caminho da solução para a distância escolhida [12].

Assim, este caminho só será substituído por outro, caso exista o fenómeno de evaporação das feromonas e o caminho não seja mais a solução para a fonte de alimento. O problema que será exposto aqui seria em reencontrar o caminho ótimo após uma alteração do percurso solução ou da fonte de alimento ou da inexistência de ambos [12]. Uma das soluções poderia ser aplicada a utilização de formigas artificiais tal como foi explicado anteriormente 2.2.

2.3.2 Pseudocódigo Representativo

O pseudocódigo pretende explicar de uma forma mais simples, ou seja, informal e simplificada, os passos deste algoritmo, usando uma combinação de linguagem natural e elementos de programação para facilitar a leitura do mesmo. O flowchart na Fig: 2.3 explica este pseudocódigo de forma visual.

O funcionamento global do algoritmo pode ser sintetizado no seguinte pseudocódigo:

```
/*
Pseudocódigo de Análise do problema ACO
-analisar distancia_fonte entre ninho e fonte;
-analisar a melhor solução;
*/
procedimento [melhor_local] = ColoniaFormigas(t_ij,t,populacao)
    // Definição das variáveis principais
    t_ij = 0          // nível de feromona acumulado para cada aresta/arco percorrido
    melhor_local      // armazena a melhor localização encontrada
    t = 0             // posição/estado inicial
    distancia         // distância percorrida para diferentes caminhos
    min_distancia = melhor_local    // menor distância encontrada melhor solução
    max_distancia = distancia       // maior distância: não é a melhor solução
    populacao = 50    // número de formigas definido inicialmente
    distancia_fonte // distancia ninho fonte

    // Inicializar variáveis para a exploração
    iniciar (distancia_fonte), (t), (t_ij);

    // Avaliar as diferentes posições até que todas tenham sido visitadas
    enquanto existirem posições a serem avaliadas (t != 0)
        // Verifica se a posição atual é a melhor possível
        se (distancia_fonte == min_distancia);

            então:
                (t_ij) != 0;          // aumenta trilha de feromona
                (melhor_local) = (min_distancia);
            se-não:
                (t_ij) = 0;           // trilha de feromona evaporado
                retorna (-1); // retorna a verificação!
        fim-se:
            // passar para a próxima posição caso seja aletardo distancia_fonte
            atualizar (t), (distancia_fonte);
    fim-enquanto:
    // Quando finalizar, retorna o melhor local encontrado
    retornar (melhor_local);
fim-procedimento
```

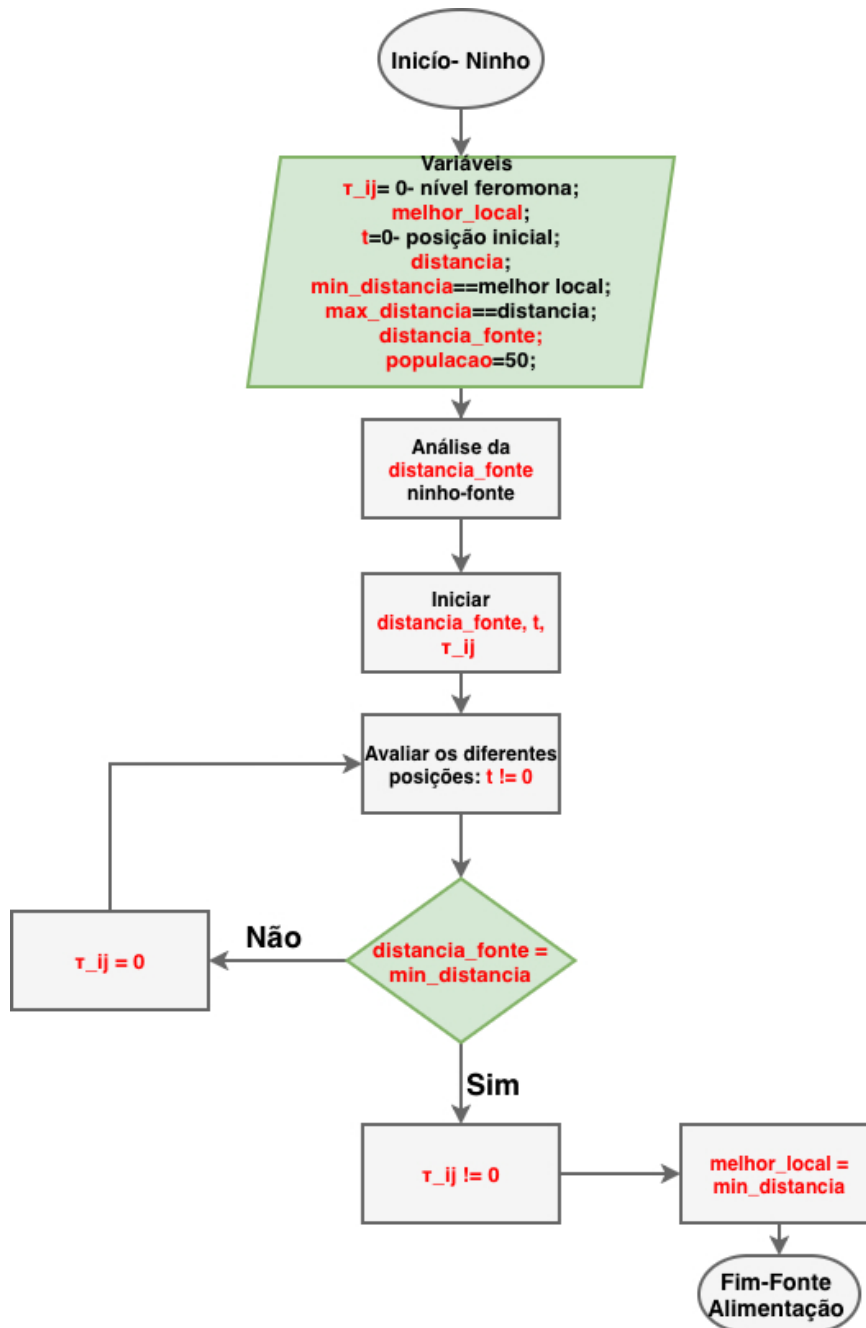


Figura 2.3: "Flowchart":explicação do pseudocódigo [11], [1].

2.4 (TSP)

O ACO é aplicado em problemas combinatórios complexos tal como o TSP entre outros. As soluções que pretendem resolver este problema são os algoritmos desenvolvidos tais como : AS e ACS , referenciados nas próximas subsecções 2.4.1 e 2.4.2.

Este problema tenta determinar a menor rota para percorrer uma série de cidades (visitando uma única vez cada uma delas sem existir repetição), retornando há cidade de origem, percorrendo o menor caminho possível, reduzindo o tempo/distância necessário para a trajetória [13].

2.4.1 AS

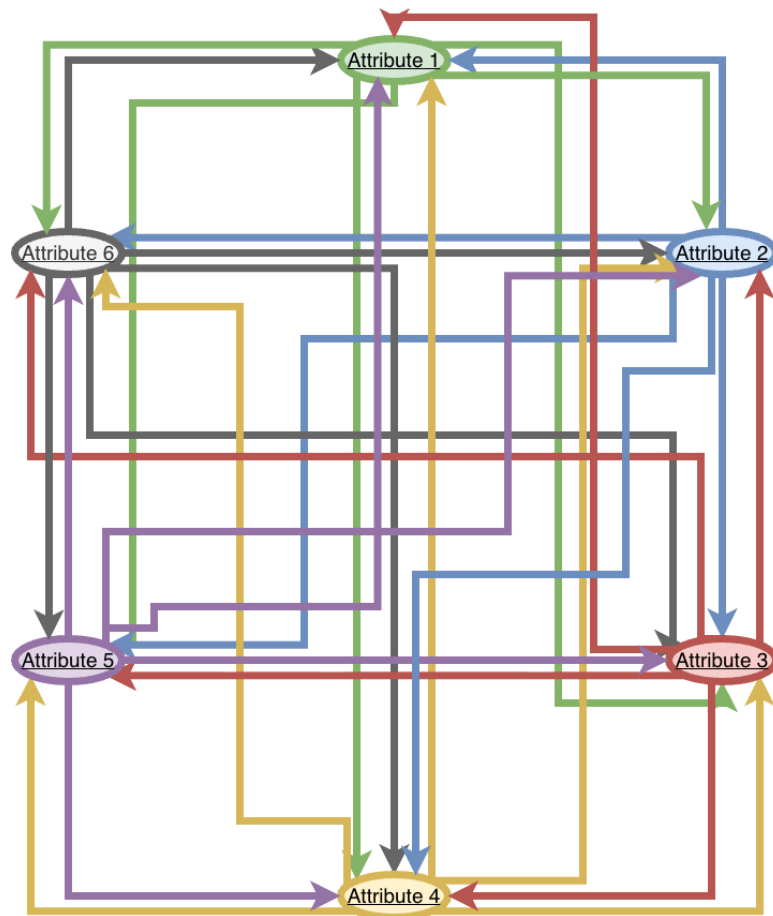
O AS, foi o primeiro algoritmo, criado em 1996, é uma das soluções para o problema do TSP até os dias de hoje!

Desenvolvido pelo autor mais citado neste relatório, Marco Dorigo [4]. "Marco Dorigo é um cientista da computação italiano, que trabalha com inteligência artificial, é diretor de pesquisa do Fundo Belga para Pesquisa Científica e codiretor do IRIDIA, o laboratório de inteligência artificial da Université Libre de Bruxelles"[2, 12].

As principais etapas que pretendem descrever este algoritmo são as seguintes:

1. **Parâmetros iniciais e feromona:** Define-se: **I:** o número de formigas, a que se chama de número da população referente aos parâmetros de controlo (α, β, ρ); **II:** número máximo de iterações, e **III:** os valores iniciais do rastro de feromona τ_{ij} . Normalmente, as feromonas iniciam com um valor constante τ_0 , distribuído de forma uniforme sobre todos os possíveis percursos.

Um exemplo esquemático de todos os possíveis recursos iniciais seria o seguinte na Fig em baixo 2.4:



Qual é o caminho mais curto?

Aqui existe um problema, porque cada caminho têm diferente tamanho, sentido e direção. Assim, fica muito difícil de saber qual é o melhor caminho a percorrer

Figura 2.4: TSP Assimétrico [11] [1].

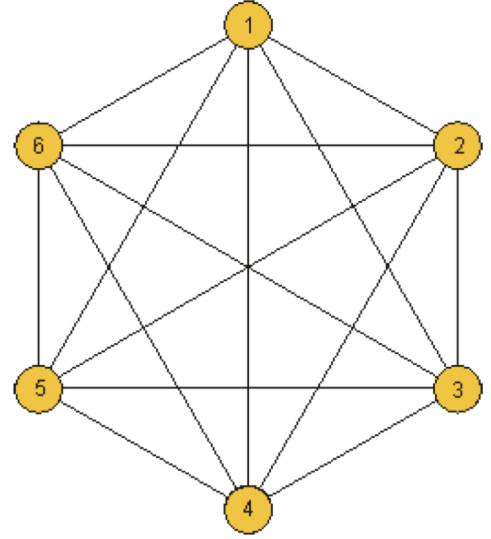
2. **Construção de soluções pelas formigas:** No esquema em baixo explicativo é possível verificar soluções para os melhores caminhos a percorrer 2.5

Dados:

- $V = \{v_1, \dots, v_m\}$ conjunto de cidades (nós)
- $A = \{(r, s) \mid r, s \in V\}$ conjunto de arestas (caminhos entre cidades)
- c_{rs} = custo associado à aresta (r, s)

Propósito:

- O caixeiro deve visitar todas as cidades
- Passando uma só vez em cada cidade
- Com o menor custo possível



TSP simétrico

Figura 2.5: Escolha do melhor caminho (TSP) exemplo já implementado![2].

Cada formiga é colocada num ponto inicial ou em um nó (por exemplo, uma cidade no TSP). Neste caso de 1 a 6.

As formigas constroem soluções de forma incremental, selecionando os próximos passos de acordo com uma probabilidade que combina a informação de feromona e uma função heurística [12].

A probabilidade P_{ij}^k de uma formiga k mover-se do nó i para o nó j é definida por:

$$P_{ij}^k = \frac{(\tau_{ij})^\alpha (\eta_{ij})^\beta}{\sum_{l \in N_i^k} (\tau_{il})^\alpha (\eta_{il})^\beta} \quad (2.1)$$

onde a explicação é a seguinte:

- τ_{ij} representa a intensidade da feromona na ligação (i,j) ;
- η_{ij} é a informação heurística (ex: inverso da distância) [8]

$$\eta_{ij} = \frac{1}{d_{ij}} \quad (2.2)$$

- α e β controlam a importância relativa destes dois fatores τ_{ij} e η_{ij} ;
- N_i^k é o conjunto de movimentos disponíveis para a formiga k a partir do nó i . No esquema em baixo explicativo é possível verificar soluções para os melhores caminhos a percorrer na Fig 2.6:

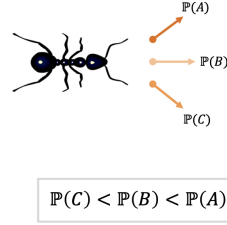


Figura 2.6: Possíveis movimentos da formiga $P(x)$, [10].

$P(A), P(B), P(C)$, representam o conjunto N_i^k , para onde o somatório do mesmo é realizado apenas sobre as possíveis soluções não repetindo os nós i já existentes pelas formigas k .

3. **Avaliação das soluções:** Após todas as formigas completarem as suas soluções, ou seja, os caminhos, avalia-se a qualidade de cada uma através da função objetivo do problema (ex: distância total do percurso). Identifica-se a melhor solução encontrada (solução ótima) na iteração e, ou opcionalmente, a melhor solução global até o momento. Após cada formiga construir uma solução (um ciclo completo, no caso do TSP), a distância total do percurso (função objetivo) para a formiga k é definido por:

$$L_k = \sum_{(i,j) \in S_k} d_{ij} \quad (2.3)$$

onde:

- S_k é o conjunto ordenado de arestas visitadas pela formiga k (percurso construído);
- d_{ij} é a distância/custo associado entre os nós (cidades) i e j [12, 10].

No fim de cada iteração:

- A **melhor solução da iteração** é determinada por:

$$L_{\text{best iter}} = \min_k L_k \quad (2.4)$$

- A **melhor solução global até o momento** pode ser atualizada por:

$$L_{\text{global best}} = \min(L_{\text{best anterior}}, L_{\text{best iter}}) \quad (2.5)$$

O caminho correspondente ao menor L é armazenado como a solução ótima até há ultima verificação. Todo este processo é essencial para o ciclo de atualização das feromonas e procura dos melhores percursos.

4. **Atualização das feromonas:** Os rastros de feromona são atualizados com base nas soluções construídas. Parte da feromona existente evapora, e uma nova quantidade é depositada nas rotas pertencentes às melhores soluções [12], [8].

O processo é descrito pela seguinte fórmula:

$$\tau_{ij} \leftarrow (1 - \rho)\tau_{ij} + \sum_{k=1}^m \Delta\tau_{ij}^k \quad (2.6)$$

onde:

- ρ é a taxa de evaporação que está entre os valores: $]0,1[$;
- se a formiga k usou o arco (i,j) ; caso contrário, $\Delta\tau_{ij}^k = 0$;

$$\Delta\tau_{ij}^k = \frac{Q}{L_k} \quad (2.7)$$

- Q é uma constante que regula a quantidade de feromona depositado;
- L_k é o custo (ou comprimento) da solução encontrada pela formiga k .

5. **Critério de paragem:** O algoritmo termina quando é atingido o número máximo de iterações, ou quando as feromonas convergem, isto é, quando a maioria das formigas passa a escolher a mesma rota indicando estabilidade da solução.

2.4.2 ACS

O ACS foi uma evolução do AS em 1997 por Marco Dorigo [4]. Dorigo, motivou o desenvolvimento de uma nova meta-heurística de otimização através da sua publicação [5]. Este é uma meta-heurística de otimização, com parâmetros principais que controlam o comportamento das formigas artificiais na busca por soluções ótimas no TSP 2.2.

Dorigo e colaboradores verificaram que o AS original, embora inovador, tendia a convergir de forma relativamente lenta e, em alguns casos tendia a soluções sub-ótimas devido a um controlo menos rigoroso da atualização de feromona. Esta limitação motivou a definição de novas regras de decisão e de atualização de trilhos de feromona, resultando no ACS como meta-heurística mais robusta e eficiente.

Assim, como uma evolução direta do AS, mantendo a ideia central de múltiplas formigas cooperativas mas adiciona estratégias mais complexas de exploração. Em particular, o algoritmo passa a utilizar regras de escolha de arestas e de atualização de feromona desenhadas para intensificar a busca nas regiões mais promissoras do espaço de soluções.

Um exemplo de uma tabela comparativa onde está presente um caso de uso comparativo é a seguinte Fig 2.7 [5].

TABLE I
A COMPARISON OF LOCAL UPDATING RULES. FIFTY-CITY PROBLEMS AND OLIVER30 WERE STOPPED AFTER 2500 ITERATIONS, WHILE RY48P WAS HALTED AFTER 10000 ITERATIONS. AVERAGES ARE OVER 25 TRIALS. RESULTS IN BOLD ARE THE BEST IN THE TABLE

	ACS			Ant-Q			ACS with $\Delta\tau(r,s)=0$			ACS without local-updating		
	average	std dev	best	average	std dev	best	average	std dev	best	average	std dev	best
City Set 1	5.88	0.05	5.84	5.88	0.05	5.84	5.97	0.09	5.85	5.96	0.09	5.84
City Set 2	6.05	0.03	5.99	6.07	0.07	5.99	6.13	0.08	6.05	6.15	0.09	6.05
City Set 3	5.58	0.01	5.57	5.59	0.05	5.57	5.72	0.12	5.57	5.68	0.14	5.57
City Set 4	5.74	0.03	5.70	5.75	0.04	5.70	5.83	0.12	5.70	5.79	0.05	5.71
City Set 5	6.18	0.01	6.17	6.18	0.01	6.17	6.29	0.11	6.17	6.27	0.09	6.17
Oliver30	424.74	2.83	423.74	424.70	2.00	423.74	427.52	5.21	423.74	427.31	3.63	423.91
ry48p	14,625	142	14,422	14,766	240	14,422	15,196	233	14,734	15,308	241	14,796

Figura 2.7: Variantes de ACS e resultados respetivos [5].

A comparação é evidente em diferentes variantes do ACS, focando especificamente nas regras de atualização local de feromona e no impacto que estas têm na qualidade das soluções para TSP de 30 a 50 cidades incluindo problemas de TSP simétricos e assimétricos.

A atualização global de feromona no ACS aplicada apenas às arestas pertencentes ao melhor caminho conhecido (por exemplo, o melhor desde o início da execução), reforça fortemente esse conjunto de arestas. Nesta atualização, o nível de feromona em cada aresta selecionada sofre evaporação e recebe depois um incremento inversamente proporcional ao comprimento do melhor caminho, concentrando o nível de feromona nas soluções mais curtas, como representado nas Fig anteriores 2.1 e 2.5.

Capítulo 3

Aplicação em Otimização [14]

Existe vários casos de estudo referente há utilização deste algoritmo em estudo. Ambos têm um objetivo que é encontrar a melhor solução possível, resolvendo o TSP.

O caso de estudo que será analisado é uma aplicação em python que consiste em resolver o problema TSP com base neste algoritmo de estudo ACO: [14].

Explicação - Algoritmos Implementados :

I- ACS: O algoritmo principal que constrói soluções com formigas artificiais 2.2, que escolhem caminhos com base em um estado de transição "regra gulosa" que equilibra exploração de novas rotas e exploração do conhecimento acumulado via feromona.

II- Algoritmo Elitista: Uma variação de ACS que reforça a feromona das melhores soluções encontradas mais fortemente do que as demais, acelerando a convergência para soluções de qualidade.

III- Algoritmo de Máximo e Mínimo: Variante que restringe os valores de feromona dentro de limites máximo e mínimo, regulando a aprendizagem para evitar que o algoritmo fique preso em soluções pobres e acelerando a convergência em fases posteriores.

3.1 Parâmetros e "Dataset" Implementado

Parâmetros:

- "colony-size": Número de formigas na colônia (padrão 15).
- "eps": Número de iterações que o algoritmo executa (padrão 50).
- "mode": Escolhe qual variante do algoritmo de ACO executar: ACS, 'Elitist' ou 'MaxMin'.
Nest caso a variante realizada foi o ACS.

Dataset:

O "dataset" fornecido é real, com 225 cidades do subcontinente Indiano. Os ficheiros incluem distâncias euclidianas, distâncias por estrada, tempo de viagem e coordenadas geográficas (latitude e longitude) das cidades.

3.2 Resultados:

Após executar o caso de estudo que está inserido na base de dados do repositório é possível observar em baixo na Fig [14], 3.1:

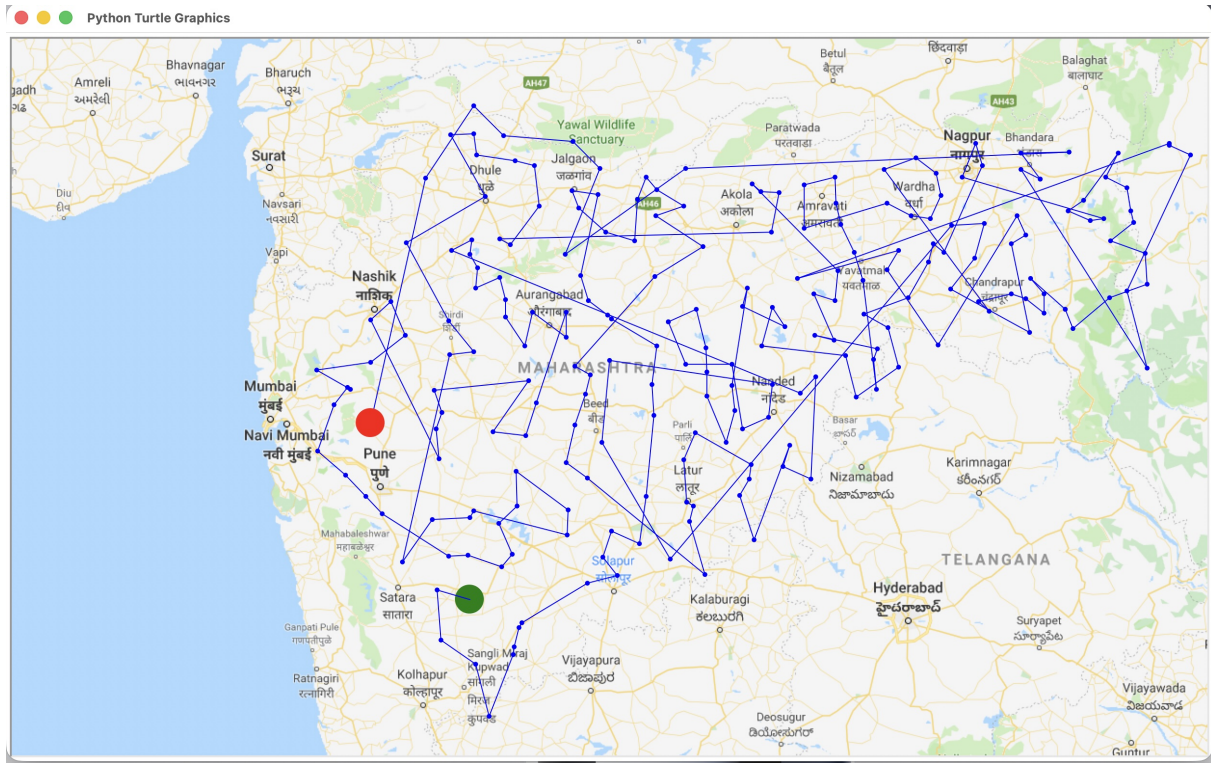


Figura 3.1: 225 cidades de "North-West and Central part of the Indian sub continent". [14].

A origem e o destino a cada execução do projeto é sempre aleatório sendo que o método de uso que está a ser sempre implementado é o ACS. Cada ponto azul é uma cidade incluindo os pontos verde e vermelho sendo no total 225 pontos referentes às 225 cidades.

Assim, com este caso de estudo podemos verificar que é possível resolver o problema do TSP.

Capítulo 4

Conclusão

Ao longo deste trabalho foi possível compreender o comportamento das formigas, estudado em diversas experiências biológicas, pode ser traduzido em algoritmos computacionais capazes de resolver problemas complexos de otimização.

Neste relatório foi analisado o comportamento biológico das formigas, com particular foco nos mecanismos de comunicação por feromonas, nos processos de "feedback" positivo e negativo e na forma como a inteligência de enxame permite a estas colônias encontrar caminhos eficientes de forma descentralizada. Estes princípios naturais serviram como base para a compreensão da estigmergia e da forma como o ambiente atua como mediador de informação, fazendo a ponte de ligação entre a biologia e o desenvolvimento do ACO.

Também, foram estudadas as principais etapas deste algoritmo, incluindo a construção probabilística das soluções pelas formigas artificiais, o papel da heurística e o processo de atualização dos níveis de feromona. O relatório abordou as variantes AS e ACS, salientando as diferenças nas regras de decisão e na forma como cada uma reforça ou evapora feromonas. Por fim, analisou-se a aplicação prática destes conceitos ao problema TSP, utilizando um "dataset" real e avaliando o comportamento das variantes do algoritmo na procura de soluções eficientes.

Conclui-se que os algoritmos inspirados na natureza, como o ACO, constituem uma abordagem altamente eficaz para resolver problemas de otimização combinatória. A análise do comportamento biológico das formigas demonstrou que mecanismos simples, como o reforço e a evaporação de feromonas, permitem alcançar soluções de elevada qualidade. Foi possível verificar que o ACO consegue adaptar-se ao ambiente e convergir progressivamente para caminhos mais eficientes.

As variantes AS e ACS, estudadas ao longo do trabalho, apresentam diferenças significativas: enquanto o AS apresenta maior simplicidade, o ACS é mais robusto e rápido na convergência graças ao seu processo de atualização mais seletivo e controlado. A aplicação prática ao problema TSP confirmou a utilidade e a versatilidade destes algoritmos, demonstrando que, mesmo em "datasets" reais e complexos, o ACO é capaz de criar soluções viáveis e de boa qualidade.

Este trabalho permitiu consolidar o entendimento teórico do algoritmo e confirmar a sua eficiência em casos práticos, destacando a importância das meta-heurísticas como ferramentas

poderosas para problemas complexos de otimização.

Referências

- [1] E. Junqueira, “Github - junqueiradeveduardo repositório.” <https://github.com/JunqueiraDevEduardo/MEECAgoritmosGeneticos>. [Citado nas páginas 1, 4, 6, 8 e 9]
- [2] imeunicampbr, “Journal ant colony optimization.” <https://www.ime.unicamp.br/chico/mt852/formigas>, 2009. [Citado nas páginas 4, 2, 9 e 10]
- [3] S. Goss, S. Aron, J.-L. Deneubourg, and J. Pasteels, “Self-organized shortcuts in the argentine ant. *naturwissenschaften* 76: 579-581,” *Naturwissenschaften*, vol. 76, pp. 579–581, 12 1989. [Citado na página 2]
- [4] C. da Wikipedia, “Marco dorigo criador do algoritmo genético.” <https://pt.wikipedia.org/wiki/Marco-Dorigo>, 2021. [Citado nas páginas 2, 9 e 12]
- [5] M. Dorigo, V. Maniezzo, and A. Coloni, “The ant system: Optimization by a colony of cooperating agents,” *IEEE Transactions on Systems, Man, and Cybernetics, Part B*, vol. 26, no. 1, pp. 29–41, 1996. [Citado nas páginas 4 e 12]
- [6] “Fork github.” <https://github.com/JunqueiraDevEduardo/col-nia-formigas-caixairo-viajante.git>. [Citado na página 4]
- [7] F. Breve, *Otimização por Colônia de Formigas (Ant Colony Optimization)*, 2025. Material didático disponível em <https://www.fabriciobreve.com/material/cin/CIN-11-ACO.pdf>. [Citado nas páginas 4 e 5]
- [8] F. De Economia, “Ant colony optimization: a literature survey.” <https://hdl.handle.net/10216/70687>, 2012. [Citado nas páginas 5, 10 e 11]
- [9] C. da Wikipédia, “Meta-heurística.” <https://pt.wikipedia.org/wiki/Meta-heur> [Citado na página 5]
- [10] F. Azevedo, “Genetic algorithms course pdf.” [Citado nas páginas 4, 5 e 11]
- [11] E. Junqueira, “Diagramas draw.io colônia de formigas: flowcharts.” <https://app.diagrams.net/>. [Citado nas páginas 4, 6, 8 e 9]
- [12] F. Breve, “Ant colony optimization.” <https://www.fabriciobreve.com/material/cin/CIN-11-ACO.pdf>, 2024. Apostila, UNESP. [Citado nas páginas 6, 9, 10 e 11]
- [13] Wikipédia, “Problema do caixeiro-viajante — wikipédia, a enciclopédia livre.” <https://pt.wikipedia.org/w/index.php?title=Problema do caixeiroviajante&oldid=68192204>, 2024. [Online; accessed 28-junho-2024]. [Citado na página 8]
- [14] “Fork github tsp python.” <https://github.com/JunqueiraDevEduardo/TSP-ACO-Exemplo-TSP-Python>. [Citado nas páginas 2, 4, 14 e 15]